

Game Design Document — StealthSurvival

Título	StealthSurvival
Autor	Daniel Fimiani
Motor	Unreal Engine 5.6 (C++)
Género	Stealth / Infiltración top-down
Modo	Un jugador
Cubre	Parcial Motores + Parcial IA en Videojuegos (un solo proyecto)
Fecha de entrega	2026-07-06

1. Descripción

StealthSurvival es un juego de infiltración en un único nivel cerrado, con cámara top-down. El jugador controla a un intruso que debe **robar un objetivo** (un documento/ítem custodiado) y **escapar con él a la zona segura**, sin ser detectado por los guardias y las cámaras que vigilan el recinto.

No hay combate frontal: la fuerza del jugador no está en pelear sino en **leer los patrones de vigilancia**, moverse en silencio, usar el entorno como cobertura y neutralizar amenazas por la espalda cuando no queda otra. Cada guardia visto suma a una **barra de detección global**; si se llena, la partida se pierde.

Referencias de Game Design: la propuesta toma la tensión de sigilo de *Metal Gear Solid* (conos de visión, alerta progresiva, distracciones sonoras), el planteo top-down de infiltración de *Monaco / Commandos*, y la economía de riesgo de *Mark of the Ninja* (el jugador siempre sabe cuánto ruido hace y qué tan visible está).

Fantasmía del jugador: ser el fantasma que entra, toma lo que vino a buscar y desaparece sin que nadie sepa que estuvo ahí.

2. Actores

Jugador

- Intruso** (`AStealthSurvivalCharacter`): personaje controlable. Se mueve en tres registros (caminar / correr / agacharse), cada uno con su propia velocidad y **radio de ruido**. Emite estímulos de sonido y de vista que la IA percibe. Puede interactuar con el mundo, hacer eliminaciones silenciosas, arrojar distracciones y volverse invisible.

NPCs (IA autónoma)

- Guardia** (`AStealthGuardCharacter` + `AStealthAIController`): enemigo principal, gobernado por un **Behavior Tree**. Patrulla rutas por NavMesh, percibe por vista y oído, y escala su comportamiento por estados de alerta (**Unaware → Suspicious → Alerted**). Investiga ruidos, persigue al jugador y abre puertas cercanas en su camino. Su **cono de visión** cambia de color según el estado (verde / amarillo / rojo) como feedback para el jugador. Puede ser eliminado por la espalda.
- Cámara de seguridad** (`AStealthSecurityCamera`): segundo perceptor, gobernado por un **State Tree**. Mecánica de detección distinta: **solo visión**, sin oído. Barre el área en un arco fijo; al ver al jugador contribuye a la detección igual que un guardia. Puede desactivarse desde un interruptor.

Actores del mundo

- Objetivo** (`AStealthObjective`): el ítem a robar. Interactuable; al tomarlo habilita la condición de escape.
- Zona de extracción** (`AStealthExtractionZone`): la zona segura / salida. Entrar con el objetivo = victoria.
- Llave** (`AStealthKey`) y **Puerta** (`AStealthDoor`): gating de acceso; ciertas puertas requieren una llave recolectada.
- Arbusto / cobertura** (`AStealthCoverBush`) y **escondite** (`AStealthHidingSpot`): elementos del entorno que rompen la línea de visión o esconden por completo al jugador.
- Interruptor de cámara** (`AStealthCameraSwitch`): desactiva una cámara de seguridad.
- Objeto arrojado** (`AStealthThrowable`): la distracción sonora que lanza el jugador.

3. Mecánicas (implementadas en C++)

- Locomoción stealth contextual.** Caminar / correr / agacharse, cada modo con su velocidad y su **radio de ruido** propio (agachado hace poco ruido y es lento; correr es rápido pero se escucha lejos). El ruido se emite periódicamente como estímulo para la IA.
- Sistema de detección por IA.** Los perceptores (guardias y cámaras) usan **AI Perception** (vista + oído). Cada perceptor que está viendo al jugador se registra

- como "watcher"; una **barra de detección global** sube mientras haya watchers activos y baja cuando no. El estado de alerta de cada guardia escala de forma progresiva y visible.
3. **Eliminación silenciosa (takedown)**. Acercándose por detrás de un guardia (validado por producto punto de orientación y distancia de trace), el jugador puede neutralizarlo en silencio con una animación de montage. Un prompt sobre el guardia indica cuándo está disponible.
 4. **Distracción arrojadiza**. El jugador lanza un objeto que, al impactar, **emite un estímulo de sonido**. Los guardias que lo escuchan cambian a estado *Suspicious* y usan EQS para elegir un punto de inspección coherente y desplazarse a investigar, dejando huecos en su patrulla.
 5. **Coberturas y escondites (Smart Objects / entorno)**. Arbustos y escondites que el jugador usa para romper la línea de visión o esconderse por completo. La cámara del juego aplica además **fade de oclusión** para que el jugador nunca quede tapado por geometría.
 6. **Invisibilidad (GAS — punto extra)**. Habilidad activa implementada con el **Gameplay Ability System** (UGA_Invisibility): por una duración limitada el jugador no puede ser percibido por la vista de la IA. Recurso de emergencia para salir de una situación comprometida.

4. Metas y obstáculos

- Meta principal:** robar el objetivo y llevarlo a la zona de extracción.
- Sub-metas / gating:** encontrar llaves para abrir puertas cerradas, desactivar cámaras molestas, aprender las rutas de patrulla.
- Obstáculos:**

- Guardias patrullando con conos de visión y oído que reaccionan al ruido.
 - Cámaras de seguridad con visión fija en arco.
 - Puertas que requieren llave.
 - La propia barra de detección: cada error visible acerca la derrota.
- Tensión central de diseño:** velocidad vs. silencio. Correr llega antes pero hace ruido; agacharse es seguro pero lento. El jugador administra ese trade-off todo el tiempo.

5. Reglas

- El jugador **gana** si entra a la zona de extracción **teniendo el objetivo**.
- El jugador **pierde** si la **barra de detección se llena** (DetectionLevel llega a 1.0).
- La detección **sube** mientras al menos un perceptor está viendo al jugador y **baja** cuando ninguno lo ve (tasas configurables: DetectionRiseRate / DetectionFallRate).
- Un guardia eliminado queda fuera de juego (desaparece tras un lifespan).
- Un guardia solo puede ser eliminado **por la espalda** y en rango.
- Las puertas con llave no se abren sin la llave correspondiente recolectada.
- El estado de la partida es explícito: **Playing → Won / Lost** (EStealthMatchState).

6. Espacio (nivel y flujo de juego)

- Un único nivel de gameplay** (Lv1_Stealth): recinto cerrado con zonas de patrulla, mobiliario y arbustos que generan cobertura y oclusión, el objetivo custodiado, puertas con llave y la zona de extracción como zona segura.
- Flujo de juego completo:**
1. **Menú Principal** (Lv1_MainMenu , nivel separado) — botones Jugar / Salir. La transición al gameplay se hace en C++ (OpenLevel → Lv1_Stealth).
 2. **Gameplay** — el ciclo de infiltración descrito arriba, con HUD activo (UStealthHUDWidget / WBP_HUD): barra de detección, texto de objetivo e indicador de si el jugador está oculto.
 3. **Pantalla final** (UStealthEndScreenWidget) — al ganar o perder, pausa la partida y muestra el resultado con opciones **Reintentar** (recarga el nivel) o **Volver al menú**. Habilita rejugabilidad sin reiniciar la aplicación.

7. Inteligencia Artificial (detalle técnico)

Herramienta	Uso en el proyecto
NavMesh + Pathfinding	Patrullaje y persecución de guardias; adaptan ruta ante obstáculos.
Behavior Tree	Cerebro del guardia (patrulla → sospecha → alerta → persecución).
State Tree	Cerebro de la cámara de seguridad (mecánica de detección distinta).
AI Perception	Vista + oído en guardias; solo vista en cámaras.
EQS	Elección del punto de inspección tras escuchar una distracción.

Smart Object / entorno Herramienta	Coberturas y escondites usados por jugador y NPCs. Uso en el proyecto
Teams (GenericTeamAgent)	Distingue jugador de IA para la percepción hostil.
Patrón Observer	UStealthAlertSubsystem difunde el estado de alerta entre las IAs (multicast).

Los agentes son **autónomos desde el inicio**, nunca quedan estáticos, y reaccionan de forma **clara y progresiva** a los estímulos. El sistema funciona con **múltiples instancias activas** en simultáneo.

8. Arquitectura y buenas prácticas

- **Gameplay Framework:** AStealthSurvivalGameMode (reglas de victoria/derrota y Tick de detección) + AStealthSurvivalGameState (estado replicable: detección, objetivo, llaves, match state) + PlayerController dedicado.
- **Comunicación entre clases:** interfaces (IInteractable , IAISightTargetInterface , IGenericTeamAgentInterface), delegados multicast (Observer del subsistema de alerta) y referencias validadas.
- **Enhanced Input:** todas las acciones (mover, mirar, correr, agacharse, takedown, arrojar, interactuar, invisibilidad) mapeadas con Enhanced Input.
- **Animación:** UStealthAnimInstance con estados diferenciados (Idle / Walk / Run / Crouch).
- **GAS:** habilidad de invisibilidad como punto extra.
- **Patrón de diseño:** Observer vía UStealthAlertSubsystem .

9. Git y control de versiones

Proyecto versionado en Git (repositorio **StealthSurvival**), con **Git LFS** para los assets binarios de Unreal. El plugin RiderLink se mantiene fuera del repo (cada dev lo instala localmente). Assets acotados a lo estrictamente usado (estilo low-poly) para mantener el repo liviano dentro del presupuesto de LFS.

10. Diversión

El corazón divertido de StealthSurvival es la **tensión del casi-me-ven**: leer un cono de visión, calcular si me da el tiempo, tirar una distracción en el momento justo y colarme por el hueco que dejó el guardia. El jugador siempre tiene información clara (ruido, visibilidad, color del cono, barra de detección) para tomar decisiones informadas, y el juego premia la paciencia y la lectura del entorno por sobre los reflejos. La invisibilidad y el takedown dan la válvula de escape cuando el plan se rompe, manteniendo la sensación de estar siempre al borde pero nunca sin salida.